

	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30723
	Apellidos y nombres del estudiante: Caiza Carlos, Chiriboga Kerlly, Villacrés Alexander

ANTECEDENTES

- El restaurante Bocatto Valley presenta problemas relacionados con la gestión manual de pedidos, generando errores en la toma de órdenes, retrasos en la comunicación entre cocina y salón y dificultades para conocer el estado real de los pedidos.
- Para solucionar esta problemática se propone el desarrollo de un sistema de gestión de pedidos basado en una arquitectura de tres capas: Presentación, Lógica de Negocio y Datos.
- Debido a la complejidad en la creación de pedidos y la necesidad de reutilizar promociones y combos predefinidos, se plantea la utilización de los patrones de diseño creacionales Builder y Prototype.
- La aplicación de estos patrones permitirá mejorar la escalabilidad, mantenibilidad y reutilización del software, reduciendo además la deuda técnica del proyecto.

OBJETIVO

• OBJETIVO GENERAL

Diseñar una arquitectura de software para el sistema Bocatto Valley utilizando los patrones creacionales Builder y Prototype con el fin de optimizar la creación de pedidos y la reutilización de promociones dentro del sistema.

• OBJETIVOS ESPECÍFICOS

1. Implementar el patrón Builder para la construcción paso a paso de los pedidos realizados desde mesa, teléfono o código QR.
2. Implementar el patrón Prototype para la clonación eficiente de promociones y combos previamente configurados.
3. Mantener una separación clara entre las capas de Presentación, Lógica de Negocio y Datos.
4. Mejorar la mantenibilidad y escalabilidad del sistema mediante el uso de patrones de diseño.

DESARROLLO

Arquitectura Propuesta

La arquitectura se encuentra organizada en tres capas principales:

○ Capa de Presentación

Responsable de la interacción con los usuarios del sistema.

Componentes:

- VistaPedido
- VistaAdministracion

Funciones:

- Registro de pedidos.
- Consulta de promociones.
- Administración de productos y combos.

- **Capa de Lógica de Negocio**

Responsable de implementar las reglas de negocio y los patrones de diseño.

Componentes:

- DirectorPedido
- PedidoBuilder
- ServicioPedido
- RegistroPrototipos
- PromocionPrototype

Funciones:

- Construcción incremental de pedidos.
- Validación de información antes de registrar pedidos.
- Gestión y clonación de promociones.

- **Capa de Datos**

Responsable de almacenar y recuperar información.

Componentes:

- RepositorioPedido
- RepositorioPromocion
- Base de Datos
- Entidad Pedido
- Entidad Promocion

Funciones:

- Persistencia de información.
- Consulta y almacenamiento de pedidos.
- Gestión de promociones registradas.

Aplicación del Patrón Builder

El patrón Builder se utiliza para construir pedidos complejos de manera incremental.

Proceso:

1. El usuario selecciona productos.
2. El DirectorPedido coordina la construcción.
3. PedidoBuilder agrega productos, cantidades y observaciones.
4. El método build() valida la información.

5. Se genera un objeto Pedido completamente construido.

Beneficios:

- Evita constructores con demasiados parámetros.
- Centraliza validaciones.
- Facilita el mantenimiento del sistema.

Aplicación del Patrón Prototype

El patrón Prototype se utiliza para gestionar promociones y combos.

Proceso:

1. El administrador registra promociones base.
2. Las promociones se almacenan en RegistroPrototipos.
3. Cuando se requiere una nueva promoción, se realiza una clonación.
4. La copia generada puede modificarse sin afectar el prototipo original.

Beneficios:

- Reduce tiempo de creación de objetos.
- Disminuye carga computacional.
- Facilita la reutilización de configuraciones existentes.

o Diagrama de Arquitectura

```
@startuml
    U2T2_Arquitectura_Builder_Prototype
    title Arquitectura del Sistema Bocatto - U2T2\nArquitectura en tres capas con Builder y Prototype
```

```
left to right direction
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam rectangleRoundCorner 12
skinparam componentStyle rectangle
skinparam packageStyle rectangle
```

```
actor "Cliente /\nMesero /\nAdministrador" as Usuario
```

```
rectangle "Capa de Presentación" as CAPA_PRESENTACION #D9EAF7 {
```

```
    rectangle "VistaPedido\n(Formulario / QR / Teléfono)" as VistaPedido #EAF4FB
```

```
    rectangle "VistaAdministración\n(Productos y Promociones)" as VistaAdmin #EAF4FB
```

```
}
```

```
rectangle "Capa de Lógica_Negocio" as CAPA_NEGOCIO #DFF2E1 {
```

```

rectangle "DirectorPedido\n(Builder Director)" as Director #FFF2CC

rectangle "PedidoBuilder\n(Builder)" as Builder #FFE6CC

rectangle "ServicioPedido" as ServicioPedido #F3FBF4

rectangle "RegistroPrototipos\n(Prototype Registry)" as Registro #FCE5CD

rectangle "PromocionPrototype\n(Prototype)" as Prototype #F9CB9C
}

rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {

    rectangle "RepositorioPedido" as RepoPedido #FAF1FC

    rectangle "RepositorioPromocion" as RepoPromo #FAF1FC

    database "Base de Datos\nPedidos / Productos / Promociones" as BD #FFFFFF

    rectangle "Pedido" as Pedido #FCF7FD

    rectangle "Promocion" as Promocion #FCF7FD
}

Usuario --> VistaPedido : registrar pedido
Usuario --> VistaAdmin : administrar promociones

VistaPedido --> Director : crear pedido
Director --> Builder : construcción paso a paso
Builder --> Pedido : build()

VistaAdmin --> Registro : solicitar clonación
Registro --> Prototype : clone()

Prototype --> Promocion : genera copia

ServicioPedido --> RepoPedido : guardar/listar
Registro --> RepoPromo : consultar prototipos

RepoPedido --> BD
RepoPromo --> BD

RepoPedido --> Pedido
RepoPromo --> Promocion

note bottom of Builder #FFF2CC
Construye pedidos de forma
incremental (mesa, QR,

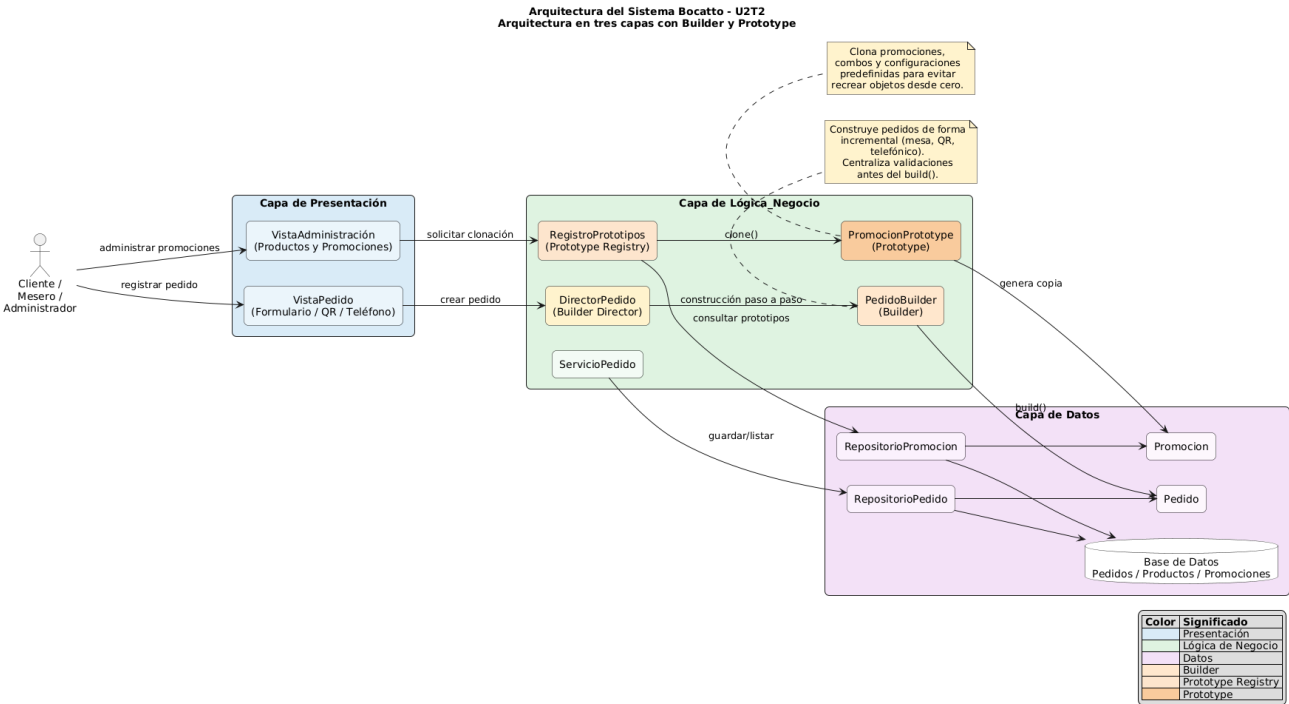
```

```
telefónico).
Centraliza validaciones
antes del build().
end note

note bottom of Prototype #FFF2CC
Clona promociones,
combos y configuraciones
predefinidas para evitar
recrear objetos desde cero.
end note

legend right
|= Color |= Significado |
|<#D9EAF7>| Presentación |
|<#DFF2E1>| Lógica de Negocio |
|<#F3E0F7>| Datos |
|<#FFE6CC>| Builder |
|<#FCE5CD>| Prototype Registry |
|<#F9CB9C>| Prototype |
endlegend

@enduml
```



CONCLUSIONES

- 1. El patrón Builder permite construir pedidos de manera segura y controlada, garantizando la integridad de la información antes de su almacenamiento.
- 2. El patrón Prototype optimiza la gestión de promociones mediante la clonación de configuraciones previamente definidas.

3. La arquitectura en tres capas facilita el mantenimiento, la escalabilidad y la reutilización de componentes.
4. La solución propuesta contribuye a reducir errores operativos y mejorar la eficiencia del restaurante Bocatto Valley

RECOMENDACIONES

1. Mantener separadas las responsabilidades de cada capa de la arquitectura, para cumplir algunos de los principios SOLID.
2. Utilizar repositorios para evitar que la vista acceda directamente a la persistencia de datos.
3. Considerar la incorporación de nuevos patrones de diseño en futuras mejoras del proyecto, para mejorar la estructura.

Sangolquí, a 09 de junio de 2026